

UPPMAX Instruction

for Language Technology Students

27.04.2026

Uppsala Universitet
Department of Linguistics and Philology

Contents

Acknowledgement	1
1. Introduction	2
1.1. Working with Linux	2
1.1.1. Common Commands	2
1.1.2. Users, Groups and Permission	3
1.1.3. Editors	3
1.2. UPPMAX	5
1.3. The new cluster: Pelle	5
1.3.1. What's New	5
1.3.2. Hardwares	5
2. Moving around	7
2.1. Login to Pelle	7
2.1.1. Setting up TOTP	7
2.1.2. SSH key	7
2.2. Projects on UPPMAX	9
3. Files Transfer and Synchronisation	10
3.1. scp	10
3.2. rsync	10
4. Softwares	12
4.1. Module System	12
4.2. Installing Binary	12
4.3. Compiling	12
5. Python Virtual Environment	14
5.1. python-venv	14
5.2. conda	14
5.3. uv	14
5.3.1. Installation	14
5.3.2. Using venv	14
5.3.3. Packages	15
6. Scheduling Jobs on Pelle	16
6.1. Nodes	16
6.2. Interactive Session	16
6.3. Slurm	16
6.4. Time Management for Jobs	16
7. Storage Management	17
7.1. Available Storages	17
7.2. Storage Usage	17
7.3. Cache	17
8. Using Jupyter Lab or Notebook	18
8.1. UPPMAX Lab	18
8.2. Running on an Interactive Session	18
8.2.1. Request an interactive session	18
8.2.2. Run Notebook in the interactive session	18
8.2.3. SSH forward	18
9. Git	20

Acknowledgement

TBD

1. Introduction

Example: Tip & Note

Blocks in normal colours are general tips or notes. Commands under these blocks should be safe to run.

Example: Warning

Blocks in red and yellow are marked as *warning*, which will include things that you might need to pay special attention to. These things might be dangerous, regarding to data, or information security.

Command Formats

For the main text, I will use \$ to indicate a command for Linux, and PS> for a command for Windows (PS means PowerShell).

1.1. Working with Linux

I feel it is important to notice that I am not intending to force people to follow this session, as everyone should choose whatever that make them feel the best. The aim of this session is to provide an alternative or a reference when you cannot use your favourite tools for some reasons and have to switch to these basic but powerful tools.

Linux is not a specific system, but a family of systems based on the Linux kernel. A specific system from the family is called a **Linux distribution**, or **distro** for short. The distro runs on Futurum is [Ubuntu](#), while for UPPMAX, it becomes [Rocky Linux](#). Different distributions provide different features.

However, as a normal user, we don't and won't need to care about this. No matter how they change, they will all include the same set of softwares, allowing us to switch without any problems.

1.1.1. Common Commands

It is not meaningful to list all of them, as we don't need to be the admin for the system. Table 1 lists some of the useful commands for LT students, and clicking the commands will lead you to their tldr pages for the usages.

Common Utilities	Handy Utilities	Text Utilities
<code>cd</code>	<code>df</code>	<code>wc</code>
<code>ls</code>	<code>du</code>	<code>head</code>
<code>cat</code>	<code>file</code>	<code>tail</code>
<code>mkdir</code>	<code>kill</code>	<code>grep</code>
<code>rm</code>	<code>less</code>	<code>sort</code>
<code>cp</code>	<code>tee</code>	<code>uniq</code>
<code>mv</code>	<code>touch</code>	<code>cut</code>
<code>find</code>	<code>pwd</code>	<code>split</code>

Table 1: Some useful commands

Tip

If you want to know the usage of a command, you can always use `man` to read the manual page, or search the command at tldr.sh if you feel it too long.

1.1.2. Users, Groups and Permission

TBD

1.1.3. Editors

Note

Unfortunately, it is impossible for me to provide an instruction for using VSCode in *Pelle* as I am not using it. However, unlike *Rackham* and *Snowy*, VSCode is available for coding in *Pelle* according to [this guide](#) on UPPMAX documentation.

Popular editors for CLI include **Vi/Vim**, **Emacs**, and **Nano**.

1.1.3.1. Vi/Vim

One of the main characters of the editor war. Vim stands for **Vi IM**proved, and you can also try [Neovim](#) or other Vi-like editor if you want some other features, but most of them share the same basic operations.

Note

- Under **Normal mode**, you can move around by pressing `h j k l` or `← ↓ ↑ →`.
- Press `a` or `i` will switch the editor to **Edit mode**, where you can start typing.
- Press `Esc` in Edit mode will bring you back to Normal mode.
- Quit the editor by pressing `:wq` in Normal mode

```

nvim :nano config
21 == Editors
20
19 Unfortunately, as neither Johan nor I use VSCode, we cannot provide a detailed
18 instruction for using VSCode in Pelle. However, unlike Rackham and Snowy, it is
17 available to connect your VSCode to Pelle according to
16 #link("https://docs.uppmax.uu.se/software/vscode_on_pelle/")[this guide] on UPPMAX
15 documentation.
14
13 Popular editors for CLI include *Vi/Vim*, *Emacs*, and *Nano*.
12
11 == Vi/Vim
10
9 *Vim* is the 'IMproved' version of *Vi*, but the operation is similar.
8
7 #rect(fill: rgb("#d8dee9"), stroke: 1pt, width: 100%)[
6 *Tip* \
5 #rect(fill: rgb("#e2eff4"), stroke: 1pt, width: 100%)[
4 You can also try #link("https://neovim.io/")[Neovim] if you want more extensibility
3 and other features.]
2 ]
1
88 #figure(
1 image("figs/nvim_in_terminal.png", width: 75%),
2 caption: [Neovim editing Typst file in terminal]
)
C @ + main.typ
:wg

```

Figure 1: Editing a file with Neovim in terminal

1.1.3.2. Emacs

Emacs is another main character of the editor war. Usually Emacs has its own interface as Figure 2, but it can be run inside a terminal as Figure 3 also. Although Emacs has commands also, its editing style will be more relied on shortcuts.

Note

- C means the Control/Ctrl key and M means the Meta key, which becomes Alt/Option for common keyboards nowadays.
- Type M-x to input command
- Use C-g for aborting a command
- Quit Emacs with C-x C-c

```

File Edit Options Buffers Tools Python Help
def load_index(json_path):
    # Open the file at 'json_path' and load the JSON data
    with open(json_path, "r") as f:
        d = json.load(f)
    return d

def search_term(index, term):
    # Convert the search term to lowercase
    term = term.lower()
    # Return the list of document IDs if the term is in the index
    if term in index.keys():
        return index[term]
    else:
        return None

if __name__ == '__main__':
    # Check that a file path was given as a command-line argument
    # Load the inverted index using the given path
    json_file = sys.argv[1]
    load_index(json_file)

U++ lab1_2025_in_search_inverted_index.py 2k L19 Git-trunk (Python EIDoc)
ESC- (<f1> for help)

```

Figure 2: Emacs with its default window

```

File Edit Options Buffers Tools Help
Welcome to GNU Emacs, one component of the GNU/Linux operating system.

Get help      C-h (Hold down CTRL and press h)
emacs manual  C-h p      Browse manuals  C-h i
emacs tutorial C-h t      Undo changes   C-x u
Buy manuals   C-h RET    Exit Emacs     C-x C-c
Activate minibuffer M-
('C-' means use the CTRL key. 'M-' means use the Meta (or Alt) key.
If you have no Meta key, you may instead type ESC followed by the character.)

Useful tasks:
Visit New File      Open Home Directory
Customize Startup   Open _scratch_ buffer

GNU Emacs 27.2 (build 1, x86_64-redhat-linux-gnu, GTK+ Version 3.24.31, cairo version 1.17.4)
of 2025-11-03
Copyright (C) 2021 Free Software Foundation, Inc.

GNU Emacs comes with ABSOLUTELY NO WARRANTY; type C-h C-w for full details.
Emacs is Free Software—Free as in Freedom—so you can redistribute copies
of Emacs and modify it; type C-h C-c to see the conditions.
Type C-h C-o for information on getting the latest version.

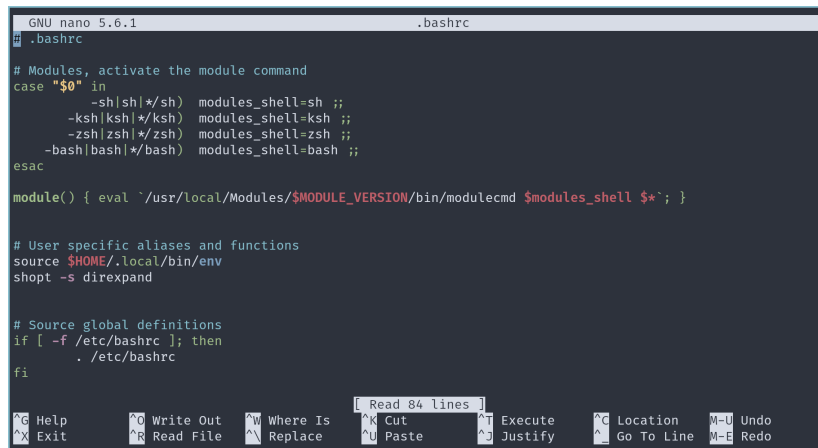
GNU Emacs 27.2 (build 1, x86_64-redhat-linux-gnu, GTK+ Version 3.24.31, cairo version 1.17.4)
of 2025-11-03
Copyright (C) 2021 Free Software Foundation, Inc.
GNU Emacs comes with ABSOLUTELY NO WARRANTY; type C-h C-w for full details.
Emacs is Free Software—Free as in Freedom—so you can redistribute copies
of Emacs and modify it; type C-h C-c to see the conditions.
Type C-h C-o for information on getting the latest version.

```

Figure 3: Emacs running inside a terminal

1.1.3.3. Nano

Nano is a light-weight editor originated from **Pico**, which is still available on macOS. It is considered easier to use as it prints some common shortcuts by default. Its light-weight design determines that it will not be as powerful as Vim and Emacs, but still for some people, if they have some ideas or just two lines of code to write, instead of waiting Vi or Emacs, they will open Nano to start.



```
GNU nano 5.6.1 .bashrc
# Modules, activate the module command
case "$0" in
  -sh|sh|*/sh) modules_shell=sh ;;
  -ksh|ksh|*/ksh) modules_shell=ksh ;;
  -zsh|zsh|*/zsh) modules_shell=zsh ;;
  -bash|bash|*/bash) modules_shell=bash ;;
esac

module() { eval `usr/local/Modules/$MODULE_VERSION/bin/modulecmd $modules_shell $*`; }

# User specific aliases and functions
source $HOME/.local/bin/env
shopt -s direxand

# Source global definitions
if [ -f /etc/bashrc ]; then
  . /etc/bashrc
fi

^G Help  ^O Write Out  ^W Where Is  [ Read 84 lines ]  ^K Cut  ^T Execute  ^G Location  M-U Undo
^X Exit  ^R Read File  ^\ Replace  ^U Paste  ^J Justify  ^_ Go To Line  M-E Redo
```

Figure 4: Editing a file with Nano in terminal

Note

Some people might feel these hints not obvious. These symbols in Nano mean:

- M is the Meta key as said above
- The ^ means the Control\Ctrl key, so ^X means you can exit Nano by pressing C-x

1.2. UPPMAX

Although *Pelle* is the only available cluster to LT students, it is still worth to mention that some concepts from the old handbook, like the chapter about memory, can still be applied despite the design differences between *Pelle* and *Rackham/Snowy*. They are general enough that can be applied to *Futurum*, and even to your future works.

As for UPPMAX itself, just a reminder, don't forget the [UPPMAX documentation](#) for guides for all functions of UPPMAX, and the [system usage chart](#) shows the current system load for each cluster.

1.3. The new cluster: Pelle

1.3.1. What's New

As we all know, the old clusters, *Rackham* and *Snowy*, were [retired](#) earlier this year, so the old days, when we would usually log in to *Rackham* and send heavy or long-time job to *Snowy*, were gone.

For students from Stockholm, besides UPPMAX, you may also try [Dardel](#), which is a similar system maintained by KTH. However, *Dardel* has a slightly different design, so it would be better to refer to [their documentation](#) for more specific information.

1.3.2. Hardwares

As mentioned above, the *Haswell* nodes are from the old *Snowy*. It is suggested to use them as a backup when all other nodes are occupied¹.

- CPUs
 - ▶ AMD EPYC 9454P 48-Core Processor 2.75 GHz

¹But sometimes even nodes on *Haswell* would be all occupied also.

- AMD EPYC 9124 16-Core Processor 3 GHz
- (Haswell) Intel Xeon E5-2630 v3 2.4 GHz
- GPUs
 - Nvidia L40s (48GB)
 - Nvidia H100 (94GB)
 - (Haswell) Nvidia T4
- Memory + Scratch²
 - (CPU Node) 768 GiB + 1.7 TiB
 - (Fat Node) 2 TiB + 6.9 TiB
 - (Fat Node) 3 TiB + 6.9 TiB
 - (GPU Node) 384 GiB + 6.9 TiB
 - (Haswell) 256 GiB + 1.8 TiB

²Scratch is a temporary storage which will be revoked after your job gets done. We will talk about this in Section 7.

2. Moving around

2.1. Login to Pelle

There are several ways to log in to *Pelle*.

2.1.1. Setting up TOTP

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

2.1.2. SSH key

Using [SSH keys](#) is considered a more secure method for connection generally, as long as you can keep your keys safe. Specifically for UPPMAX, you can skip entering password and TOTP by using your SSH key.

Warning

The key generation processes below should be processed **on the client**, which in this case, is **your own computer**. Even the server should **NOT** have your private key.

2.1.2.1. Creating the key

The default options for `ssh-keygen` should be enough for normal users. However, it is recommended to use Ed25519 for generating keys, as the command below, unless you have a super old computer. I personally suggest setting a different filename for distinguish, and set a passphrase for unlock the key.

```
$ ssh-keygen -t ed25519 -C "<Comment on the key>"
```

```
~  
$ ssh-keygen -t ed25519  
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/home/kunihiro/.ssh/id_ed25519): test_ed25519  
Enter passphrase for "test_ed25519" (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in test_ed25519  
Your public key has been saved in test_ed25519.pub  
The key fingerprint is:  
SHA256:e2p6uT205VY8fh07w/Am7U06h0v2b+yIyv9nPGLO2qc kunihiro@fedora-x1c  
The key's randomart image is:  
+--[ED25519 256]--+  
|  
|  
| S . |  
| .. ..+o |  
|.o.+ =0++ |  
| +=o BBo#0 |  
|.o.+**E600 |  
+--[SHA256]-----+
```

Figure 5: Generating a key located in `./test_ed25519`

ssh-keygen will generate two files for you: one ends with .pub is the **public key** and another one is the **private key**. Figure 5 here is only for demonstration, and the key had been revoked after finishing the instruction, but for you, **DO NOT LEAK YOUR PRIVATE KEY**.

2.1.2.2. Upload the key

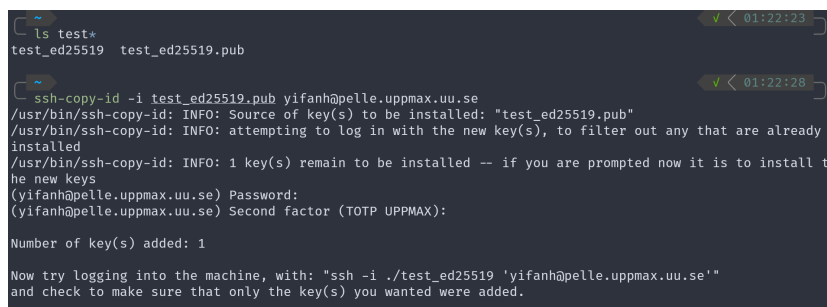
We need to upload the public key to UPPMAX. The easiest way is to use ssh-copy-id. Replace <PATH_OF_PUB_KEY> to the actual path of your public key.

```
$ ssh-copy-id -i <PATH_OF_PUB_KEY> <USERNAME>@pelle.uppmx.uu.se
```

For Windows, there is no ssh-copy-id, so one has to use a slightly weird method: read the contents of the public key, and pipe it to ssh to send it to server.

```
PS> cat <PUB_KEY> | ssh <USERNAME>@pelle.uppmx.uu.se "mkdir -p .ssh && tee .ssh/authorized_keys"
```

This command should work for Windows 10 and newer versions.



```
ls test*
test_ed25519  test_ed25519.pub

ssh-copy-id -i test_ed25519.pub yifanh@pelle.uppmx.uu.se
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "test_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
(yifanh@pelle.uppmx.uu.se) Password:
(yifanh@pelle.uppmx.uu.se) Second factor (TOTP UPPMAX):

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i ./test_ed25519 'yifanh@pelle.uppmx.uu.se'"
and check to make sure that only the key(s) you wanted were added.
```

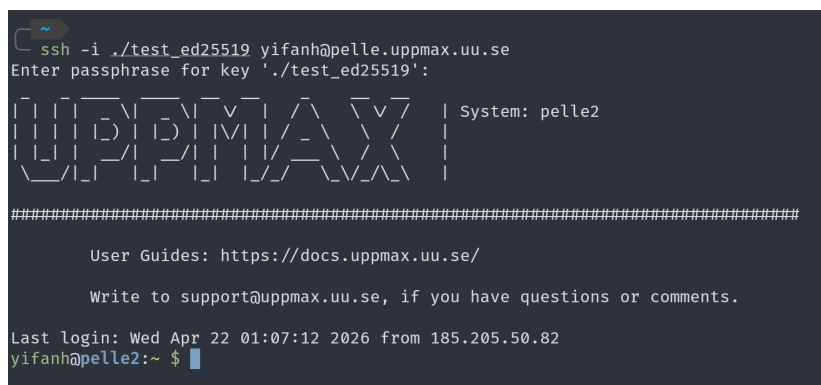
Figure 6: Uploading the key to UPPMAX

2.1.2.3. Login with key

Now you can log in to UPPMAX by specifying your private key. Replace <PATH_OF_PRIV_KEY> and <USERNAME> to your private key path and your username for login.

```
$ ssh -i <PATH_OF_PRIV_KEY> <USERNAME>@pelle.uppmx.uu.se
```

As Figure 7, will be prompted to input the passphrase of the key, but not the password and TOTP for UPPMAX.



```
ssh -i ./test_ed25519 yifanh@pelle.uppmx.uu.se
Enter passphrase for key './test_ed25519':

#####
System: pelle2

User Guides: https://docs.uppmx.uu.se/

Write to support@uppmx.uu.se, if you have questions or comments.

Last login: Wed Apr 22 01:07:12 2026 from 185.205.50.82
yifanh@pelle2:~ $
```

Figure 7: Using the key to log in UPPMAX

2.1.2.4. SSH config

By adding the key to config file located in `.ssh/config`, one can login with the provided key by default.

```
Host *.uppmax.uu.se
    IdentityFile <PATH_OF_KEY>
```

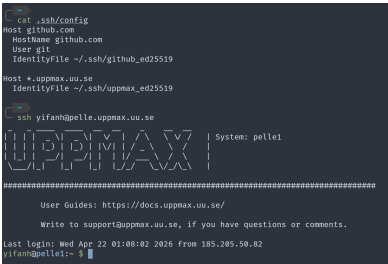


Figure 8: SSH config example

2.2. Projects on UPPMAX

Projects

Active Projects You Belong To

Click on the project name in the leftmost column to see more project information.

Project	PI	Project Title	Project Type	Centre	Start Date	End Date	Your Role
UPPMAX 2026/1-95	Sara Stymne	Computational Linguistics @ UU	Centre Local	UPPMAX	2026-04-01	2027-04-01	member
UPPMAX 2026/1-123	Meriem Beloucif	SLN711 & SLN718: Machine Translation x 2	Centre Local	UPPMAX	2026-04-08	2027-01-01	member
UPPMAX 2025/2-505	Eva Pettersson	SLN709 Master's Thesis in Langua	Centre Local Compute	UPPMAX	2026-01-01	2026-12-01	member
UPPMAX 2025/3-5	Sara Stymne	SLN71X - Masterprogram i språkteknologi	Centre Local Compute	UPPMAX	2025-03-25	2026-05-01	member

Request Membership in Project

To request membership in a project, use the search function below to search for the project. You may use project name/Dnr, project title or the name of the principal investigator.

Search for Project

Past Project You Belonged To

Click on the project name in the leftmost column to see more project information.

Project	PI	Project Title	Project Type	Centre	Start Date	End Date	Your Role
UPPMAX 2020/2-2	Joakim Nivre	CL@UU	Centre Local Compute	UPPMAX	2020-03-27	2026-02-01	member

Figure 9: Projects page of SUPR

3. Files Transfer and Synchronisation

3.1. scp

In case you forget, you can always use scp for transferring files between the client and the server.

```
$ scp <SOURCE> <DESTINATION>
```

If you want to transfer a folder, use -r option like what we normally do for cp.

```
$ scp -r <SOURCE> <DESTINATION>
```

For example, you want to upload a file call model.pth to a folder called my_models under your UPPMAX home directory (So it is ~/my_models/), then you want to move the folder back to the current directory of your local machine, you can do this on your machine:

- To upload, do:

```
$ scp my_model.pth <USERNAME>@pelle.uppmax.uu.se:my_models
```

- To download, do (don't forget the dot in the end, which means current directory):

```
$ scp -r <USERNAME>@pelle.uppmax.uu.se:my_models .
```

3.2. rsync

Note

Windows does NOT ship rsync by default, so you need to have either WSL or MinGW or Cygwin installed

[rsync](#) is a utility for transferring and synchronising files like scp, but it provides more advanced features like excluding files, reduces latency and so on.

rsync can be used for transferring files like `rsync source target` as scp, but more commonly it is used for synchronising, so the basic operations are more related to a directory like:

- `$ rsync -a source_dir target_dir` for copying a directory, and
- `$ rsync -a source_dir/ target_dir` for all contents in the directory.

Warning

Pay attention to the final slash when using rsync.

- Without the slash, the folder itself will be included for transferring
- With the slash, the folder will not be included. Instead, you will transfer all your files in the folder

If you are not sure about what you typed, use options -anv for a dry-run and verbosed output.

Using the same example above, we have:

- To synchronise all contents from a local folder called `my_models/` to server, do:

```
$ rsync -av my_models/ <USERNAME>@pelle.uppmax.uu.se:my_models
```

- To do it in opposite direction but with the directory also:

```
$ rsync -av <USERNAME>@pelle.uppmax.uu.se:my_models .
```

4. Softwares

Usually for Linux, there should be a package manager available to install new softwares. However, as we don't have the permission for such tasks, we have to rely on other methods to install softwares.

4.1. Module System

The module system used by UPPMAX is called [Lmod](#), but for convenience, it is wrapped as the module command in UPPMAX. You can find all available modules [here](#)³.

- To **search a module**, use `$ module spider <MODULE_NAME>`

Tip

You can search a module with RegExp with the `-r` option as:

```
$ module -r spider <EXPRESSION>
```

- To **load a module**, use `$ module add <MODULE_NAME>`

Note

The system distinguishes the upper cases and the lower cases, which means that when you want to load a module called `FFmpeg`, it will complain if you write `ffmpeg`.

- To **unload all modules**, use `$ module purge`
- To **list loaded modules**, use `$ module list`

4.2. Installing Binary

However, there are still chances that a module is unavailable on the system. While you can send an email to [UPPMAX support](#), you can download the binary and run it locally.

4.3. Compiling

It is a rare situation, but if there is no binary file provided, you can try to compile things yourself.

There is no a common method to build a software, as there are so many toolchains for different usages and different languages. Thus, this session is more like a reminder that this way exists, and you can follow the provided guide (if any) to build the software.

³This table list softwares on *Bianca*, but all of them should be included in *Pelle* also.

Building

Configure the build using CMake:

```
cmake -Bbuild -DCMAKE_INSTALL_PREFIX=/usr
```

NOTE: The `-DCMAKE_INSTALL_PREFIX` option above will install the files to the `/usr` directory, instead of the default `/usr/local` location. For more information on available options, you can run:

```
cmake -Bbuild -LH
```

To use a different compiler, or compiler flags, you can specify these before the `cmake` command. For example:

```
CC=clang CFLAGS=-Wextra cmake -Bbuild -DCMAKE_INSTALL_PREFIX=/usr
```

The `espeak-ng` and `speak-ng` programs, along with the `espeak-ng` voices, can then be built with:

```
cmake --build build
```

The data (language dictionaries, phoneme tables, intonation) can be built with:

```
cmake --build build --target data
```

Specific languages can be compiled by running the built binary directly:

```
ESPEAK_DATA_PATH='pwd' /build build/src/espeak-ng --compile=LANG
```

Figure 10: I once had to compile the software myself following the provided guide.

5. Python Virtual Environment

5.1. python-venv

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

5.2. conda

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

5.3. uv

[uv](#) is one of my favourite tool for doing Python stuff, as most of the time I don't need modified packages from conda-forge, and it is super fast. You can give it a try if conda runs slow.

5.3.1. Installation

uv provides a standalone installer. You don't need a computing node for the installation.

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

5.3.2. Using venv

The management of venv with uv is more similar to the original venv. One can create a venv by running:

```
$ uv venv <VENV_NAME> --python <PY_VER>
```

The Python version is optional. If not set, it will use the latest version. The venv can be activated with the original style:

```
$ source <VENV_NAME>/bin/activate
```

And deactivated with


```
$ deactivate
```

5.3.3. Packages

uv is not intended to be an exact clone of pip, but swapping out `pip install` for `uv pip install` should work most of the time. For example:

```
$ uv pip install torch datasets transformers
```

The `uv pip` inherits most options of the original `pip`. For example, you can get a list of packages by running:

```
$ uv pip freeze
```

6. Scheduling Jobs on Pelle

6.1. Nodes

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.2. Interactive Session

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.3. Slurm

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

6.4. Time Management for Jobs

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7. Storage Management

7.1. Available Storages

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7.2. Storage Usage

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

7.3. Cache

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

8. Using Jupyter Lab or Notebook

8.1. UPPMAX Lab

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequaeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

8.2. Running on an Interactive Session

If you just need a temporary space for testing some idea, I strongly suggest using the so-called [UPPMAX Lab](#)⁴. It provides some pre-built environments for running temporary, ephemeral, non-persistent works, as all data will be lost when the environment is deleted, and an environment will expire after two days.

But if you somehow do require a full node to run notebook, here are the steps.

8.2.1. Request an interactive session

Login as usual, and start an interactive session.

```
$ interactive -A <PROJ_NAME> -p <NODE> -t <TIME>
```

Pay attention to the name of node that allocated to you, as we will use the name later. The name starts with 'p' followed by some digits.

8.2.2. Run Notebook in the interactive session

You can also specify a port if necessary by setting `--port=<PORT_NUM>`.

```
$ jupyter notebook --no-browser --ip=0.0.0.0
```

8.2.3. SSH forward

Now open another terminal, connect to UPPMAX with port forwarding option added: `-L <PORT>:<HOST>:<HOSTPORT>`.

`<PORT>` and `<HOSTPORT>` is the one you set before for Jupyter Notebook, by default 8888, and `<HOST>` is the name of node that mentioned before. For example, if your port is set to 2333 and node is p201, you should start a new connection by:

```
$ ssh -L 2333:p201:2333 <USERNAME>@pelle.uppmx.uu.se
```

Then you can click the link with the address `127.0.0.1` from Jupyter to open Notebook from your local browser.

⁴Currently no official name for this service

Note

You should **NOT** terminate any connections until you finish using Notebook.

9. Git

TBD